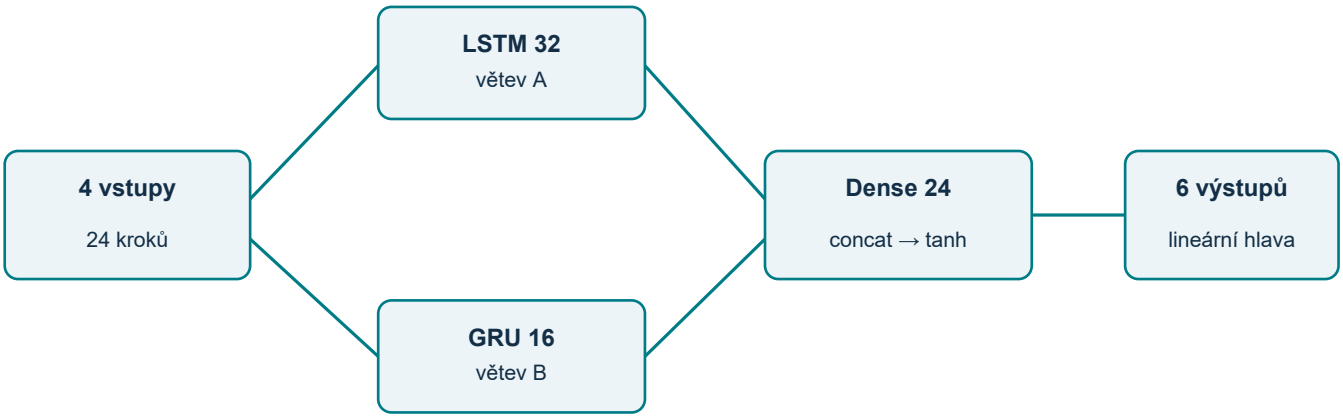
MQL5 GPU Library

Technický a uživatelský manuál • verze 2.3.0

Grafové neuronové sítě pro finanční časové řady



DLL pro MetaTrader 5 a samostatné aplikace ve Windows. Instalace pro IC Trading MT5, konfigurace topologie, datový kontrakt, více trhů, přímá predikce horizontů, checkpointy a úplná reference C ABI.

Autor projektu: Tomáš Bělák / RemindCZ

Rozsah: podporované acyklické grafy; 45 exportů; CUDA float32; double na hranici API.

Technické ověření a kvalita tržní predikce jsou dvě samostatná měření. Tato verze neobsahuje prokázaný výsledek na živém účtu IC Trading.

Obsah

1. Účel a rozsah 3

2. Požadavky a obsah balíčku 4

3. Instalace do IC Trading MT5 5

4. Graf sítě a konfigurace 6

5. Textový zápis topologie 7

6. ABI, tenzory a numerika 8

7. Trénink, souběh a životní cyklus 9

8. Checkpointy a reprodukovatelnost 10

9. Data, cíle a kontrola úniku budoucnosti 11

10. Křivka budoucnosti a provoz indikátoru 12

11. Parametry indikátoru 13

12. Samostatný projekt časových řad 14

13. Sestavení a ověřování 15

14. Diagnostika a doporučený provoz 16

15. Zdroje a licence 17

16. Úplný přehled exportů 18

1. Účel a rozsah

Knihovna poskytuje GPU výpočty rekurentních neuronových sítí pro 64bitový MetaTrader 5 i samostatné aplikace ve Windows. Verze 2.3.0 přidává grafové topologie, vrstvy Dense a Identity, více horizontů predikce, indikátor využívající více trhů a klienta pro Python bez dalších Python balíčků.

Dodávka obsahuje DLL, C hlavičku a importní knihovnu, sdílené MQL5 deklarace, zdrojový a zkompilovaný indikátor, syntetický smoke test, Python příklady, instalační skript a tento manuál. Zdroje a testy umožňují sestavení z konkrétní revize. Číselná správnost knihovny není důkazem výnosnosti predikcí.

Libovolnost topologie znamená libovolný orientovaný acyklický graf podporovaných operací: LSTM, GRU, tanh RNN, Dense a Identity. Lze volit šířky, hloubku, větvení, spojení konkatenací či součtem a reziduální cesty. Časová rekurence je uvnitř rekurentních vrstev. Konvoluce, attention, Transformer, vrstvy s libovolným uživatelským CUDA kódem, normalizace vrstev ani cykly mezi uzly nejsou součástí této verze.

Výchozím výzkumným cílem je AUDUSD na H1. Není doloženo, že jde o snadno předvídatelnou měnu. Vazba AUD na zahraniční a komoditní faktory je ekonomický důvod pro experiment, nikoli záruka predikční schopnosti. Indikátor měří kvalitu mimo trénovací období a standardně odmítá model, který nepřekoná jednoduchou referenci.

2. Požadavky a obsah balíčku

Ověřená sestava používá Windows x64, NVIDIA GeForce RTX 3060, CUDA Toolkit 13.0.48 a MSVC 19.44. CPU režim DLL není implementován. Sestavený balíček cílí na GPU architekturu 8.6; pro jiný hardware sestavte a ověřte příslušnou variantu. Přítomnost NVIDIA GPU sama o sobě kompatibilitu negarantuje.

Součást	Umístění / účel
DLL	MQL5/Libraries/MQL5GPULibrary_LSTM.dll
MQL deklarace a grafový parser	MQL5/Include/*.mqh
Indikátor	MQL5/Indicators/LSTM_CrossAsset_Forecast.mq5 a .ex5
Integrační test pro terminál	MQL5/Scripts/LSTM_SmokeTest.mq5 a .ex5
Samostatný klient	python/mql5gpu.py, python/financial_series.py
Příklady	examples/train_csv.py, examples/predict_csv.py
C/C++ integrace	include/MQL5GPULibrary.h a .lib
Instalace a integrita	tools/Install-MT5.ps1, SHA256.json

Nasazení vyžaduje kompatibilní ovladač NVIDIA a Microsoft Visual C++ 2015–2022 Redistributable x64. Tato konkrétní DLL dynamicky používá cublas64_13.dll a curand64_10.dll; cuBLAS dále vyžaduje cublasLt64_13.dll. Všechny musí pocházet z kompatibilní instalace CUDA 13. Systémové knihovny Windows a VC++ nepocházejí z projektu. Balíček nezahrnuje stovky MB runtime NVIDIA ani jejich instalátor.

3. Instalace do IC Trading MT5

Použijte datovou složku správné instance terminálu: v MT5 zvolte Soubor → Otevřít složku dat. Nezaměňujte ji s Program Files ani se složkou jiné instalace. Před výměnou již načtené DLL terminál zavřete. Instalátor nepřepisuje existující soubory bez explicitního přepínače -Force.

```
# Spusťte z rozbaleného release balíčku.
.\tools\Install-MT5.ps1 -TerminalDataPath 'C:\cesta\k\datum\terminalu' `
  -CopyCudaRuntime -CudaPath 'C:\Program Files\NVIDIA GPU Computing Toolkit\CUDA\v13.0'
```

Parametr -CopyCudaRuntime zkopíruje tři výše uvedené CUDA knihovny z vaší existující instalace do MQL5/Libraries. Bez něj zajistěte jejich dostupnost v prostředí terminálu sami. Po změně prostředí MT5 restartujte. Instalátor nepřihlašuje účet, nezískává hesla a neprovádí obchody.

1. Povolte import DLL v nastavení programu a v parametrech indikátoru nebo skriptu.

2. Spusťte LSTM_SmokeTest a zkontrolujte protokol Experts. Skript trénuje na syntetických datech; nepoužívá obchodní příkazy.

3. Otevřete graf AUDUSD H1 podle přesného názvu instrumentu u vašeho účtu IC Trading.

4. Připojte LSTM_CrossAsset_Forecast. Nastavte InpDrivers na přesné názvy symbolů z Market Watch, včetně případných přípon. Cílový instrument je vždy instrument grafu.

5. Vyčkejte na stažení historie a dokončení kandidátního modelu. Stavový text rozlišuje čekání, trénink, přijetí a zamítnutí modelu.

Výchozí prediktory jsou NZDUSD, USDJPY a USDCAD. Komodity, například XAUUSD, lze přidat nebo zaměnit v InpDrivers. Rozdílné obchodní seance však mohou způsobit chybějící společné svíčky. Program chybějící data nedomýšlí; při nedostatku souvislých oken je třeba zkrátit sekvenci nebo upravit seznam trhů. Symboly nejsou odvozeny z reklamního odkazu brokera a jejich dostupnost závisí na konkrétním účtu/serveru.

4. Graf sítě a konfigurace

Externí vstup má identifikátor -1. Uzly dostávají identifikátory 0, 1, 2 atd. v pořadí vytvoření. Každý rodič musí být externí vstup nebo dříve přidaný uzel. Všechny uzly musejí přispívat do zvoleného výstupního uzlu; zapomenutá větev je chyba. Maximálně 1024 uzlů a 64 rodičů na uzel, dále omezení celočíselných indexů a dostupné GPU paměti.

Veřejný typ	ID	Chování
LSTM	0	Skrytý a buněčný stav, standardní vnitřní brány
GRU	1	Aktualizační a resetovací brána; konvence viz kapitola 6
RNN	2	Rekurentní tanh vrstva
Dense	3	Lineární projekce a zvolená aktivace v každém čase
Identity	4	Přenos sloučeného vstupu; vhodné pro sum/concat uzly

Dense podporuje aktivaci 0=linear, 1=tanh, 2=ReLU, 3=sigmoid. Ostatní typy vyžadují parametr aktivace 0; jejich vnitřní rekurentní aktivace se tím nemění. Merge 0=concat skládá vlastnosti v pořadí rodičů. Merge 1=sum sčítá rodiče prvek po prvku a vyžaduje stejné šířky. Identity má šířku vstupu; při vytvoření lze zadat 0 a nechat ji odvodit.

```
int h = DN_Create();
int input[] = {-1};
DN_BeginGraph(h, 4, 24); // 4 vlastnosti, 24 časových kroků
int a = DN_AddGraphNode(h, 0, 32, 0, 0.1, input, 1, 0);
int b = DN_AddGraphNode(h, 1, 16, 0, 0.1, input, 1, 0);
int parents[] = {a, b};
int c = DN_AddGraphNode(h, 3, 24, 1, 0.0, parents, 2, 0);
DN_SetGraphOutput(h, c, 6); // šest výstupů lineární hlavy
// V aplikaci kontrolujte každý návratový kód, pak DN_Free(h).
```

DN_AddGraphNode vrací ID uzlu; nula je úspěch pro první uzel a -1 znamená chybu. Nevyhodnocujte jeho výsledek jako běžný booleovský kód. DN_BeginGraph zahazuje předchozí architekturu, hlavu a trénovací data. Konfigurujte nový kandidátní handle, pokud potřebujete bezpečně zachovat aktivní model.

5. Textový zápis topologie

MQL5 pomocná funkce GPUBuildTopology a Python Model.configure přijímají stejný popis. Uzly oddělujte středníkem, šest polí dvojtečkou, rodiče čárkou. Používejte desetinnou tečku; MQL parser podporuje běžný desetinný zápis dropout bez exponentu. Mezery kolem polí se odstraní.

```
typ:sirka:aktivace:dropout:spojeni:rodice
```

Povolené názvy jsou malými písmeny: lstm, gru, rnn, dense, identity; linear, tanh, relu, sigmoid; concat, sum. Dropout musí ležet v intervalu 0 až méně než 1. Výstupní lineární hlava se přidá samostatně; její počet výstupů je v indikátoru InpHorizon.

```
# Jednoduchá hlubší LSTM (při zadání odstraňte komentáře):
lstm:64:linear:0.1:concat:-1;lstm:32:linear:0:concat:0

# Dvě větve s projekcí na společný výstup:
lstm:32:linear:0.1:concat:-1;gru:16:linear:0.1:concat:-1;dense:24:tanh:0:concat:0,1

# Reziduální součet větví se stejnou šířkou:
dense:16:tanh:0:concat:-1;gru:16:linear:0:concat:0;identity:0:linear:0:sum:0,1

# Čistě dopředná varianta, poslední časový krok:
dense:32:relu:0:concat:-1;dense:16:tanh:0:concat:0
```

Dense a Identity pracují odděleně v každém časovém kroku. Lineární výstupní hlava čte poslední krok vybraného uzlu. Čistě Dense síť tedy sama nespojí časovou historii; pro využití celé sekvence vložte rekurentní vrstvu, případně předem vytvořte vlastnosti z časových zpoždění. Více výstupů představuje přímou predikci více horizontů, nikoli automatickou rekurzivní simulaci.

6. ABI, tenzory a numerika

Rozhraní je C ABI pro Windows x64 s deklarací __stdcall. Vstupy a výstupy mají typ double; výpočty na GPU používají float32. Aplikace vlastní své vstupní a výstupní buffery. Při LoadBatch se data zkopírují před návratem. Hlavička include/MQL5GPULibrary.h je autoritativní pro C/C++; MQL5 používá jediný sdílený importní soubor.

```
X[sample, time, feature]
index = (sample * sequence + time) * features + feature
in = sequence * features
X má batch * in prvků; T má batch * outputs prvků
Y má batch * outputs prvků; index = sample * outputs + output
layout = 0
```

Všechny počty musejí být kladné a součiny se musejí vejít do podporovaných indexů. Všechny hodnoty musejí být konečné a reprezentovatelné jako float32. Samotné C ukazatele nenesou skutečnou délku bufferu; volající odpovídá za správně alokovaný počet prvků. Nelze bezpečně předat zkrácené pole a očekávat, že DLL délku zjistí.

Skrytý i buněčný stav se na začátku každé sekvence nulují. Mezi voláními PredictBatch se nepřenášejí. Dropout je aktivní pouze při tréninku na výstupech uzlů, nikoli přímo na časovém skrytém stavu. Větve sdílejí výstup rodiče a jeho dropout masku. Zpětný průchod sčítá příspěvky všech spotřebitelů před propagací rodičem.

GRU používá h = (1-z) * h_previous + z * candidate a resetovací bránu po projekci skrytého stavu kandidáta. Konvence se může lišit od jiných frameworků; prostý přepis vah z cizího GRU modelu není garantovaně ekvivalentní. Import ONNX ani automatická konverze PyTorch/TensorFlow není implementována.

7. Trénink, souběh a životní cyklus

Standardní pořadí je Create → konfigurace → LoadBatch → TrainAsync → průběžné čtení stavu → PredictBatch → SaveState/GetState → Free. Každý model má vlastní handle. Operace vyžadující jeho zámek při vytížení rychle selžou. S aktivně trénovaným modelem neprovádějte predikci; použijte samostatný handle aktivního modelu, jako to dělá indikátor.

Stav GetTrainingStatus	Význam a další krok
0	Neprobíhá trénink
1	Probíhá práce; lze číst průběh nebo požádat o zastavení
2	Dokončeno, také po požadovaném zastavení; lze číst výsledek
-1	Selhání nebo neplatný handle; ihned přečtěte GetError

DN_Train je blokující, DN_TrainAsync spouští pracovní vlákno. DN_StopTraining pouze žádá o zastavení mezi minibatchi. DN_Free odebere handle, požádá o ukončení, počká na pracovní vlákno a uvolní GPU prostředky. Před unload DLL musejí být všechny handly uvolněny. Opakované Free téhož handle je bezpečné.

Optimalizátor AdamW používá learning rate a weight decay, kladný počet epoch a nepovinnou mez gradientu. Nulový target_mse vypíná předčasné ukončení podle cílové MSE. Vyhodnocení celé trénovací množiny probíhá v první epoše, každých 50 epoch, v poslední plánované epoše a při ukončení. Telemetrická MSE proto nemusí být aktualizována po každém minibatchi.

Plán learning rate začíná znovu při každém Train; Adam momenty a jeho krok pokračují mezi tréninky i po výměně dat. Obnova checkpointu nebo snapshotu Adam resetuje. GradNorm zahrnuje všechny parametry před ořezáním. Nečíselná ztráta nebo gradient vede k chybě; běh nemusí být atomicky vrácen k počátečním vahám.

ProgressAll zapisuje čtyři int výstupy a potom double v pořadí lr, mse, best_mse, grad_norm, percent, elapsed_sec, eta_sec. Jde o jednotlivě bezpečné atomické hodnoty, nikoli jediný nedělitelný snímek. Diagnostika GetError je společná pro celou DLL a jiný handle ji může přepsat. Čtěte ji bezprostředně po neúspěchu.

8. Checkpointy a reprodukovatelnost

DN_SaveState vrací potřebný počet bajtů včetně ukončovací nuly. DN_GetState zkopíruje připravený ASCII text do bufferu o alespoň této velikosti. Obě volání na jednom handle serializujte. DN_LoadState očekává platný nulou ukončený text. Python klient ukládá text bez koncové nuly; ctypes ji při předání doplní.

Sekvenční modely se ukládají jako SEQMODEL_V3. Grafové modely používají SEQMODEL_V4: přidávají počet vstupních vlastností, rodiče, merge a aktivaci každého uzlu a výstupní uzel. Čtení podporuje také LSTM_V1 a LSTM/RNN SEQMODEL_V2. Starý V2 GRU se odmítá, protože chybějí jeho kandidátní váhy.

Diskové identifikátory jsou historicky RNN=0, GRU=1, LSTM=2, Dense=3, Identity=4. Liší se od veřejných selektorů API. Checkpoint neupravujte záměnou těchto čísel podle tabulky veřejného API. Parser kontroluje rozměry, propojení, přesné počty parametrů, konečnost hodnot i konec souboru.

Neplatný checkpoint zachová aktivní model. Nové GPU vrstvy se připraví před výměnou. Úspěšné načtení vymaže stará trénovací data, snapshoty, optimizer a telemetrii. Váhy se zapisují s přesností pro float32 round-trip. Samotný checkpoint neobsahuje normalizaci, pořadí instrumentů, časové pásmo, trénovací data ani RNG stav.

Pro nasazení ukládejte společně weights.txt a metadata.json. Zachovejte symboly v pořadí, sekvenci, horizont, mean/scale, target_scale, validační metriky a časové hranice. Znovunačtení vah zachová predikce v ověřené konfiguraci, ale nový trénink nemusí být bitově reprodukovatelný. Veřejný seed API není součástí této verze.

9. Data, cíle a kontrola úniku budoucnosti

Indikátor používá pouze uzavřené svíčky. Pro každou cílovou svíčku hledá přesně stejný čas otevření na všech prediktorech. Vlastnost je logaritmický výnos mezi uzavírací cenou této a předchozí cílové svíčky. Chybějící čas nebo neplatná cena zneplatní celý řádek. Okno musí mít souvislou sérii platných řádků.

```
r[t,f] = log(close[t,f] / close[t-1,f])
x[t,f] = (r[t,f] - mean_train[f]) / std_train[f]
target[t,h] = 100 * log(close[t+h,target] / close[t,target])
forecast_close[h] = anchor_close * exp(prediction[h] / 100)
```

Mean a standardní odchylka se počítají z jedinečných trénovacích řádků, nikoli z validačního období ani z opakovaně překrytých oken. Minimální std je 1e-8. Dělení je chronologické, výchozí validační podíl 20 %. Mezi posledním trénovacím počátkem a prvním validačním počátkem je mezera větší než celý horizont, takže všechny trénovací cíle končí před prvním validačním počátkem.

Validační vstupy mohou obsahovat starší již známé řádky; to odpovídá skutečné predikci. Budoucí validační cíle ani jejich ceny se neúčastní fitování normalizace či tréninku. Opakovaný výběr topologie podle stejné validace může přesto vést k přeučení výběru modelu. Pro seriózní výzkum oddělte další nedotčené testovací období a proveďte walk-forward vyhodnocení.

Referenční model předpovídá nulový výnos, tedy nezměněnou budoucí cenu. Skóre je součet čtverců chyb modelu dělený součtem čtverců skutečných výnosů ve stejných validačních oknech a horizontech. Poměr pod 1 znamená lepší MSE než reference. Výchozí limit 0.98 vyžaduje nejméně přibližně 2% zlepšení této metriky. Jde o vybraný validační výsledek, nikoli statistický test významnosti ani čistý výnos po nákladech.

10. Křivka budoucnosti a provoz indikátoru

Timer jednou za sekundu kontroluje historii a stav kandidáta. Křivka se přepočítá po nové uzavřené svíčce nebo po přijetí modelu. U H1 nevzniká nový informační vstup s každým tickem. Predikce se proto během otevřené svíčky nedeklaruje jako nová tická prognóza. Trénink probíhá v DLL asynchronně; příprava historie a validace se provádí v událostech MQL5.

Anchor je poslední uzavřená cena. První horizont odhaduje uzavření aktuálně otevřené svíčky, další body následující svíčky. PLOT_SHIFT posune vykreslované body doprava. Horizont znamená počet budoucích obchodních svíček, nikoli zaručený počet kalendářních hodin přes víkendy či svátky. Indikátor zapne odsazení grafu doprava; při odebrání obnoví původní volbu.

Zobrazení	Interpretace
Modrá křivka	Model splnil nastavený validační limit při přijetí
Oranžová křivka	InpRequireSkill=false a model limit nesplnil
Šedá vodorovná křivka	Není přijatý model; zobrazuje se persistence
Chybějící křivka	Chybějí poslední společná data nebo výstup překročil limit
Tečkované hranice	Predikce plus/minus násobek validační RMSE pro horizont

Tečkované hranice nejsou kalibrovaný interval spolehlivosti. Násobek 1.96 sám o sobě neznamená 95% pokrytí. Při nevyhovujícím novém kandidátovi zůstává aktivní dříve přijatý model; jeho kvalita může časem klesat. Načtení indikátoru znovu vyvolá trénink, protože aktivní model indikátor zatím nepersistuje mezi relacemi.

CSV v MQL5/Files zaznamenává první vydanou predikci pro daný počátek a instanci. Obsahuje čas otevření kotevní svíčky, instrument, horizont, kotevní a predikovanou cenu, režim modelu a jeho poslední validační poměr. Historické řádky se po pozdější změně modelu nepřepisují. Do logu se může dostat persistence vydaná před dokončením kandidáta; sloupec model ji výslovně odlišuje.

11. Parametry indikátoru

Vstup	Výchozí hodnota	Význam
InpDrivers	NZDUSD,USDJPY,USDCAD	1 až 15 dalších unikátních instrumentů
InpTimeframe	H1	Musí odpovídat časovému rámci grafu
InpHistoryBars	2500	Počet uzavřených cílových svíček
InpSequence	24	Délka vstupní sekvence, 2 až 512
InpHorizon	6	Počet přímých výstupů, 1 až 64
InpTopology	LSTM32 + GRU16 → Dense24	Přesný zápis viz zdroj / kapitola 5
InpEpochs / InpBatch	60 / 64	Epochy a velikost minibatch
InpLearningRate	0.001	Počáteční krok optimalizace
InpRetrainBars	24	Interval nového kandidáta ve svíčkách
InpValidationFraction	0.20	Chronologický holdout, 0.10 až 0.40
InpRequireSkill	true	Přijmout jen kandidáta pod limitem
InpMaximumMSERatio	0.98	MSE modelu / MSE persistence
InpBandMultiplier	1.96	Násobek RMSE v log-výnosu
InpMaxAbsForecastReturn	0.10	Mez absolutního log-výnosu; výstup se skryje
InpWriteForecastCSV	true	Prospektivní CSV v MQL5/Files
InpInstance	default	Unikátní jméno: 1–32 písmen ASCII, číslic, _ či -

Indikátor omezuje history × sequence × features na 20 milionů vstupních hodnot. Nejde o záruku, že se libovolně široká síť vejde do VRAM. Při nedostatku paměti snižte šířku vrstev, délku sekvence, historii nebo minibatch. Při inicializaci se kontrolují duplicity instrumentů, dostupnost symbolů, oprávnění DLL i minimální verze 20300.

12. Samostatný projekt časových řad

Python klient používá pouze standardní knihovnu a ctypes. Vyžaduje 64bitový Python ve Windows a stejné GPU/runtime závislosti jako MT5. MetaTrader, přihlášený broker ani Python balíčky NumPy/PyTorch nejsou pro tento způsob použití potřeba. Alternativně lze volat C ABI přímo z C/C++, C# nebo jiného prostředí s odpovídajícím FFI.

Vstup je společné CSV se sloupcem time a jedním kladným konečným close pro každý požadovaný instrument. Cílový instrument je první v --symbols. Čas musí být ISO 8601 s explicitním časovým pásmem, unikátní a přísně rostoucí. Řádky předem časově zarovnejte; skript nestahuje data, nepřevzorkovává je a nerozpozná chybně označené časové pásmo brokera.

```
time,AUDUSD,NZDUSD,USDJPY,USDCAD
2026-01-05T10:00:00+00:00,0.6600,0.6050,150.00,1.3700
2026-01-05T11:00:00+00:00,0.6601,0.6051,150.02,1.3699
```

Ukázka je pouze popis formátu, nikoli skutečná tržní data. Pro trénink je nutná delší historie. Nepravidelné řádky znamenají horizont v počtu pozorování; pro hodinový horizont připravte hodinovou řadu s jasnou politikou obchodních seancí.

```
python examples/train_csv.py `
  --dll MQL5/Libraries/MQL5GPULibrary_LSTM.dll `
  --csv prices.csv --epochs 60 --output output/model

python examples/predict_csv.py `
  --dll MQL5/Libraries/MQL5GPULibrary_LSTM.dll `
  --csv recent_prices.csv --model output/model
```

Trénink vytvoří weights.txt, metadata.json a forecast.json. Metadata obsahují normalizaci naučenou pouze na trénovací části, pořadí symbolů, časové hranice a validační poměr. Samostatný příklad uloží i slabší model a ve výstupu uvede beats_persistence; příjemce musí sám uplatnit pravidlo přijetí. Inference používá uloženou normalizaci, zkontroluje rozměry modelu a neprovádí nový trénink.

13. Sestavení a ověřování

```
cmake -S . -B build -A x64 -DCMAKE_CUDA_ARCHITECTURES=86
cmake --build build --config Release --parallel 4
ctest --test-dir build -C Release --output-on-failure
powershell -NoProfile -ExecutionPolicy Bypass -File tools/compile_mql.ps1
```

Překlad vyžaduje Visual Studio 2022 C++ tools, Windows SDK, kompatibilní CUDA toolkit a CMake 3.24+. Parametr architektury zvolte podle cílového GPU. Původní řešení Visual Studio zůstává dostupné; DN_CudaVersion umožňuje volbu toolkitu. Přenosné CPU testy lze spustit s DN_BUILD_CUDA=OFF, neověřují však numeriku CUDA.

Testy pokrývají kontrolu rozměrů a přetečení, poškozené checkpointy a zachování modelu, shodu exportů C/CUDA/MQL5, chronologické dělení dat, referenční rekurence, numerické gradienty, větvení a merge grafu, vícevýstupové modely, serializaci a asynchronní ukončování. NVIDIA Compute Sanitizer kontroluje GPU přístupy. Přesný místní výsledek a omezení jsou v docs/VALIDATION.md.

MetaEditor kompiluje nový indikátor a smoke test proti sdíleným deklaracím. Úspěšný překlad EX5 není sám o sobě důkaz funkčnosti konkrétního přihlášeného brokerového účtu. Nasazení do živého grafu, kvalita dostupné historie, skutečná latence a dlouhodobá predikční schopnost vyžadují samostatný provozní test.

PDF vzniká z tohoto dokumentu skriptem tools/build_manual.py (ReportLab, pypdf pro kontrolu). Release skript tools/package_release.ps1 přidává sestavené DLL/EX5, dokumentaci a SHA256 manifest. Manifest ověřuje shodu lokálního balíčku, není digitálním podpisem vydavatele. Finální binární soubory nesměšujte s hlavičkami z jiné revize. Pokud PowerShell blokuje lokální skript, -ExecutionPolicy Bypass v uvedeném příkazu platí pouze pro spuštěný proces; použijte jej jen pro zkontrolované skripty z tohoto balíčku.

14. Diagnostika a doporučený provoz

Příznak	Kontrola / řešení
MT5 nemůže načíst DLL	x64, DLL imports, správná datová složka, CUDA a VC++ runtime
DN_Create vrací 0	Ihned GetError; ověřit GPU, ovladač a kompatibilitu binárního kódu
Unavailable broker symbol	Přesný název z Market Watch včetně přípony
Loading history	Vyčkat na stažení historie všech instrumentů
Too few synchronized windows	Délka historie, souběžné seance, kratší sekvence / jiní prediktory
Candidate rejected	Model nesplnil validační limit; šedá reference je očekávané chování
Forecast hidden	Nedostupná poslední data nebo překročená mez výnosu
Model busy	Počkat na dokončení / použít druhý handle pro inference
Neplatný checkpoint	Verze, koncová nula, úplný soubor a přesné počty parametrů
Nedostatek VRAM	Menší síť, minibatch, sekvence nebo množství dat

Začněte smoke testem, potom demo grafem a prospektivním CSV. Porovnávejte model s persistencí odděleně pro každý horizont, režim trhu a časové období. Teprve následná strategie může zahrnout spread, skluz, poplatky a rozhodovací pravidla. Indikátor žádnou strategii ani řízení rizika neprovádí. Bez měření skutečných dat nelze označit konkrétní měnu za dobře predikovatelnou.

Pokud využijete jiný cílový instrument, změňte graf, prediktory a případně hyperparametry, ale zachovejte chronologickou validaci. Změna seznamu symbolů mění význam vstupních kanálů a vyžaduje nový model a normalizaci. Návratový kód úspěšného výpočtu neznamená, že jsou vstupní ekonomické údaje správně zarovnány.

15. Zdroje a licence

Projekt: https://github.com/RemindCZ/MQL5GPULibrary_LSTM

MQL5 CopyRates: https://www.mql5.com/en/docs/series/copyrates

MQL5 DRAW_LINE: https://www.mql5.com/en/docs/customind/indicators_examples/draw_line

MetaEditor příkazový překlad: https://www.metatrader5.com/en/metaeditor/help/beginning/integration_ide

RBA, Drivers of the Australian Dollar Exchange Rate:
https://www.rba.gov.au/education/resources/explainers/drivers-of-the-aud-exchange-rate.html

Broker zadaný uživatelem: https://www.ictrading.com/en

Zdrojové kódy projektu mají licenci MIT podle LICENSE.txt. Závislosti NVIDIA, Microsoft a MetaQuotes se řídí vlastními licencemi. Tento manuál popisuje implementaci verze 2.3.0. Tržní volba a výchozí parametry slouží jako základ výzkumu, nikoli jako výsledek backtestu konkrétního IC Trading účtu.

16. Úplný přehled exportů

PDF příloha automaticky vkládá všech 45 deklarací z aktuální C hlavičky s vysvětlením návratových hodnot. Podrobný kontrakt je také v docs/API.md. Pokud není uvedeno jinak, int operace vrací 1 při úspěchu a 0 při selhání. Výjimkami jsou handle, ID uzlu, stav, počty, délka serializace a numerická telemetrie.

DN_GetVersion

```
int DN_GetVersion(void);
```

Vrací 20300 pro verzi 2.3.0. Umožňuje odmítnout nekompatibilní DLL před konfigurací.

DN_BeginGraph

```
int DN_BeginGraph(int h, int features, int sequence);
```

Zahájí prázdný graf s pevným počtem vlastností a časových kroků. Vymaže předchozí model a data.

DN_AddGraphNode

```
int DN_AddGraphNode(int h, int type, int width, int activation, double dropout, const int* parents, int
 parent_count, int merge);
```

Vrací ID uzlu 0 nebo vyšší, -1 při chybě. Typy a aktivace viz kapitola 4. parents je pole parent_count platných int.

DN_SetGraphOutput

```
int DN_SetGraphOutput(int h, int node, int outputs);
```

Vybere uzel pro poslední časový krok a vytvoří lineární hlavu s outputs výstupy. Všechny uzly musejí přispívat do výstupu.

DN_GetInputDim

```
int DN_GetInputDim(int h);
```

Vrací počet vstupních vlastností, 0 při neúplné konfiguraci nebo chybě.

DN_GetOutputDim

```
int DN_GetOutputDim(int h);
```

Vrací počet výstupů nakonfigurované hlavy, 0 bez hlavy nebo při chybě.

DN_GetSequenceLength

```
int DN_GetSequenceLength(int h);
```

Vrací délku sekvence, 0 při chybě.

DN_Create

```
int DN_Create(void);
```

Vrací kladný handle nebo 0 při selhání vytvoření CUDA prostředků.

DN_Free

```
void DN_Free(int h);
```

Blokující uvolnění handle včetně zrušení a dokončení workeru. Opakované volání je bezpečné.

DN_SetSequenceLength

```
int DN_SetSequenceLength(int h, int seq_len);
```

Nastaví kladnou délku sekvence. Změna zneplatní načtená trénovací data.

DN_SetMiniBatchSize

```
int DN_SetMiniBatchSize(int h, int mbs);
```

Nastaví kladnou velikost minibatch; ověřuje se také bezpečnost součinů rozměrů.

DN_AddLayerEx

```
int DN_AddLayerEx(int h, int in, int out, int act, int ln, double drop);
```

Původní sekvenční API. act je 0=LSTM, 1=GRU, 2=RNN; ln musí být 0. in=0 umožňuje odvození šířky. V grafovém režimu se odmítá.

DN_AddGRULayer

```
int DN_AddGRULayer(int h, int in, int out, double drop);
```

Zkratka sekvenčního API pro GRU; stejné rozměrové a dropout podmínky.

DN_AddRNNLayer

```
int DN_AddRNNLayer(int h, int in, int out, double drop);
```

Zkratka sekvenčního API pro tanh RNN; stejné rozměrové a dropout podmínky.

DN_SetGradClip

```
int DN_SetGradClip(int h, double clip);
```

Nastaví konečný nezáporný limit globální normy gradientu; 0 vypíná clipping.

DN_SetOutputDim

```
int DN_SetOutputDim(int h, int out_dim);
```

Vytvoří nebo nahradí lineární hlavu za poslední vrstvou. Zneplatní načtená data; v grafu ověří spojení všech uzlů.

DN_LoadBatch

```
int DN_LoadBatch(int h, const double* X, const double* T, int batch, int in, int out, int l);
```

Zkopíruje X a T do interního úložiště. in=sequence×features, out=šířka hlavy, l=0. Kontrolujte délky vlastních bufferů.

DN_PredictBatch

```
int DN_PredictBatch(int h, const double* X, int batch, int in, int l, double* Y);
```

Predikuje pro batch sekvencí, zapíše batch×outputs double do Y. l=0. Během tréninku téhož handle selže.

DN_SnapshotWeights

```
int DN_SnapshotWeights(int h);
```

Vytvoří interní snapshot vah, bez dat, normalizace a úplného optimizer/RNG stavu.

DN_RestoreWeights

```
int DN_RestoreWeights(int h);
```

Obnoví úplný snapshot a resetuje Adam. Bez existujícího snapshotu selže.

DN_Train

```
int DN_Train(int h, int epochs, double target_mse, double lr, double wd);
```

Blokující trénink. epochs>0, lr>0, target_mse a wd konečné nezáporné. Návrat 1 značí dokončení běhu.

DN_TrainAsync

```
int DN_TrainAsync(int h, int epochs, double target_mse, double lr, double wd);
```

Spustí worker a vrátí 1 při úspěšném zahájení. Dokončení a případné chyby ověřte přes status/result.

DN_GetTrainingStatus

```
int DN_GetTrainingStatus(int h);
```

Stavy 0/1/2/-1 jsou popsány v kapitole 7. Zastavený běh končí stavem 2.

DN_GetTrainingResult

```
void DN_GetTrainingResult(int h, double* out_mse, int* out_epochs);
```

Zapíše výslednou MSE a počet dosažených epoch. Čtěte po dokončení; ukazatele musejí být platné.

DN_StopTraining

```
void DN_StopTraining(int h);
```

Neblokující žádost o ukončení mezi minibatchi; neznamená okamžité uvolnění GPU.

DN_GetProgressEpoch

```
int DN_GetProgressEpoch(int h);
```

Aktuálně dosažená epocha. Čtěte během tréninku; při neplatném handle se vrací 0.

DN_GetProgressTotalEpochs

```
int DN_GetProgressTotalEpochs(int h);
```

Plánovaný počet epoch. Čtěte během tréninku; při neplatném handle se vrací 0.

DN_GetProgressMiniBatch

```
int DN_GetProgressMiniBatch(int h);
```

Aktuální minibatch. Čtěte během tréninku; při neplatném handle se vrací 0.

DN_GetProgressTotalMiniBatches

```
int DN_GetProgressTotalMiniBatches(int h);
```

Počet minibatchí v epoše. Čtěte během tréninku; při neplatném handle se vrací 0.

DN_GetProgressLR

```
double DN_GetProgressLR(int h);
```

Aktuální learning rate. Čtěte během tréninku; při neplatném handle se vrací 0.

DN_GetProgressMSE

```
double DN_GetProgressMSE(int h);
```

Poslední vyhodnocená trénovací MSE. Čtěte během tréninku; při neplatném handle se vrací 0.

DN_GetProgressBestMSE

```
double DN_GetProgressBestMSE(int h);
```

Nejlepší vyhodnocená trénovací MSE. Čtěte během tréninku; při neplatném handle se vrací 0.

DN_GetProgressGradNorm

```
double DN_GetProgressGradNorm(int h);
```

Poslední norma gradientu před clippingem. Čtěte během tréninku; při neplatném handle se vrací 0.

DN_GetProgressTotalSteps

```
int DN_GetProgressTotalSteps(int h);
```

Počet optimizer kroků v běhu. Čtěte během tréninku; při neplatném handle se vrací 0.

DN_GetProgressPercent

```
double DN_GetProgressPercent(int h);
```

Průběh v procentech; dokončení včetně stop nastaví 100. Čtěte během tréninku; při neplatném handle se vrací 0.

DN_GetProgressElapsedSec

```
double DN_GetProgressElapsedSec(int h);
```

Uplynulá doba v sekundách. Čtěte během tréninku; při neplatném handle se vrací 0.

DN_GetProgressETASec

```
double DN_GetProgressETASec(int h);
```

Odhad zbývajících sekund; po dokončení 0. Čtěte během tréninku; při neplatném handle se vrací 0.

DN_GetProgressAll

```
int DN_GetProgressAll(int h, int* out_epoch, int* out_total_epochs, int* out_mb, int* out_total_mb,
double* out_lr, double* out_mse, double* out_best_mse, double* out_grad_norm, double* out_pct, double*
out_elapsed_sec, double* out_eta_sec);
```

Zápis do 4 int a 7 double ukazatelů, pořadí přesně podle deklarace. Jednotlivě atomické vzorky, nikoli společný snapshot.

DN_GetLayerCount

```
int DN_GetLayerCount(int h);
```

Vrací počet vrstev/uzlů včetně existující lineární výstupní hlavy, 0 při chybě.

DN_GetLayerWeightNorm

```
double DN_GetLayerWeightNorm(int h, int l);
```

L2 norma hlavní matice W vrstvy l (od 0); index rovný počtu uzlů označuje výstupní hlavu. Nezahrnuje bias ani oddělené GRU kandidátní matice; Identity vrací 0.

DN_GetGradNorm

```
double DN_GetGradNorm(int h);
```

Vrací poslední globální normu gradientu před clippingem; dostupná také v progress.

DN_SaveState

```
int DN_SaveState(int h);
```

Připraví text checkpointu a vrátí jeho délku v bajtech včetně NUL, 0 při chybě.

DN_GetState

```
int DN_GetState(int h, char* buf, int max_len);
```

Zkopíruje předem připravenou serializaci do buf. max_len musí stačit včetně NUL. Vrací 1/0.

DN_LoadState

```
int DN_LoadState(int h, const char* buf);
```

Načte NUL-ukončený checkpoint. Neplatný soubor zachová model; úspěch resetuje data a optimizer.

DN_GetError

```
void DN_GetError(short* buf, int len);
```

Zapíše ukončenou UTF-16 zprávu do len prvků typu short; při zkrácení nejvýše len−1 znaků. Diagnostika je globální.